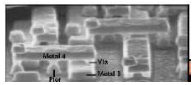
Introduction au language de description de matériel : VHDL

C. Belleudy





Université Nice SOPHIA ANTIPOLIS

Concevoir des circuits ?



➔ *Diversité des applications et complexité accrue*

➔ *Architecture : Processeur + accélérateur matériel propre à chaque application*

Pouquoi un langage de description de matériel ?

- Description performante, lisible, portable.
- Maîtriser le temps de développement.
- Pérennité des descriptions : Réutilisation des modules déjà développés et testés
- Simulations à différents niveaux de la conception

VHDL : VHSIC (Very High Speed Integrated Circuit) Hardware Description Langage

Développer par le DOD (département américain de la défense).

Langage VHDL : objectifs

- *Spécification (cahier des charges),*
- *Conception (comparaison de différents algorithmes ou solutions)*
- *Description pour la synthèse automatique*
- *Simulation fonctionnelle*
- *Vérification des contraintes temporelles*
- *Preuve formelle : vérification fonctionnelle*
- *Langage unique pour la spécification, la description, la simulation (la synthèse).*
- *Indépendance vis-à-vis du circuit cible (ASIC, FPGA …)*
- *Langage assurant une portabilité des outils,*
- *Langage Hiérarchique Top-down*

➔ *Exemple : traitement vidéo, compression /décompression …*

Simulation/Synthèse VHDL

Création de modèles de simulations :



Création d'un circuit intégré :

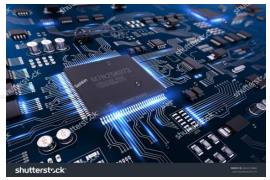


Le simulateur *: interprète le langage VHDL, comprend l'ensemble du langage*
Le synthétiseur *: traduit la description VHDL en netlist logique, comprend uniquement un sous ensemble du langage*

Plan

1 – Les différents types de description en VHDL
2 – Syntaxe de base
4 – Les instructions concurrentes
5 – Les instructions séquentielles

Comment décrire un composant ?



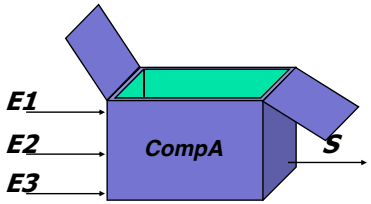
Un composant = entité (ENTITY).

1 - Description externe

- son nom,
- ses entrées et ses sorties.
...



2 - Description interne au composant



Comment décrire un composant : la syntaxe

➢ Un composant = **entité (ENTITY)**.

Description externe :

- son nom,
- définition des paramètres génériques,
- ses entrées et ses sorties.




```
ENTITY nom_du_composant IS
    GENERIC ( paramètre1 : type_du_paramètre := valeur_par_défaut;
              paramètre2 : type_du_paramètre := valeur_par_défaut;
              ...);
    PORT (  nom_signal, ... : direction type := initialisation ;
            ...
            nom_signal, ... : direction type );
END nom_du_composant ;
...
```

Description externe : syntaxe

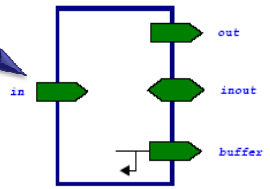
ENTITY nom_du_composant *IS*

bit (0,1), bit_vector, std_logic (0,1, X, U, …), std_logic_vector …

PORT (nom_signal, … : direction type := initialisation ;
…
nom_signal, … : direction type);

Rmq : après la dernière déclaration, pas de ;

END nom_du_composant ;
…



Comment décrire un composant : exemple

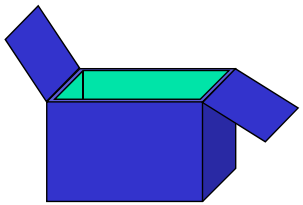


ENTITY CompA *IS*
PORT (E1 : *IN* std_logic ;
E2 : *IN* std_logic ;
E3 : *IN* std_logic ;
S : *OUT* std_logic) ;
END CompA; …
Ou
ENTITY CompA *IS*
PORT (E1,E2,E3 : *IN* std_logic ;
S : *OUT* std_logic) ;
END CompA; …
Ou E : *IN* std_logic_vector(3 downto 1);

Donner la description externe d'un additionneur 4 bits puis 32 bits, d'une bascule D puis JK, d'un registre 32 bits.

Comment décrire un composant : Description interne

…
ARCHITECTURE nom_architecture *OF* nom_du_composant *IS*
<<Déclaration des composants internes>>
<<Déclaration des types>>
<<Déclaration des signaux>>
BEGIN
<< zone de description du composant>>
END nom_architecture;



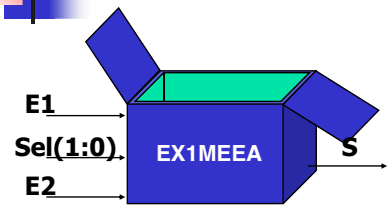
Description flot de données (data flow)
S = a+b

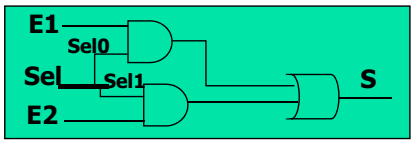
Description structurelle (structural)

Description comportementale (behavioral)
Ou Process

Combinaison de plusieurs types de descriptions

Description flot de donnée : exemple

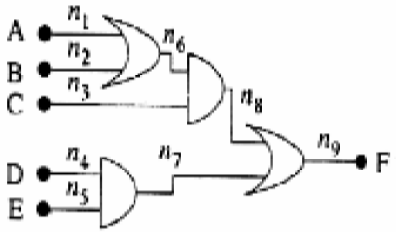




Description flot de donnée : équivalent au niveau RTL (Register Transfert Langage), les signaux passent à travers des couches d'opérateurs logiques qui décrivent les étapes successives qui font passer des entrées d'un module à sa sortie.

ENTITY EX1MEEA *IS*
PORT (E1,E2 : *IN* std_logic ;
Sel : *IN* std_logic_vector(1 downto 0) ;
S : *OUT* std_logic) ;
END EX1MEEA;
ARCHITECTURE arch *OF* EX1MEEA *IS*
BEGIN
S <= (E1 *AND* SEL(0) *OR* (E2 *AND* SEL(1));
END arch;

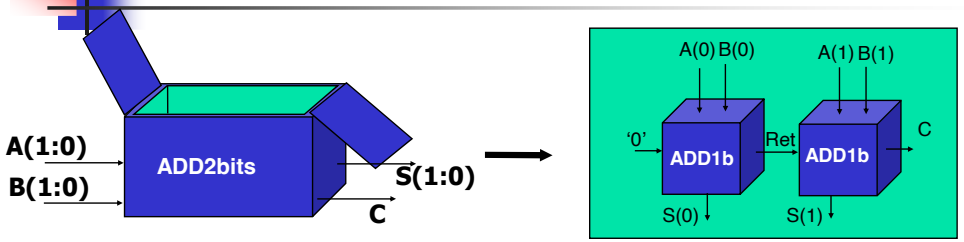
Description flot de données



Donner la description VHDL de ce composant.

Donner la description d'un additionneur 1 bit cascadable.

Description structurelle

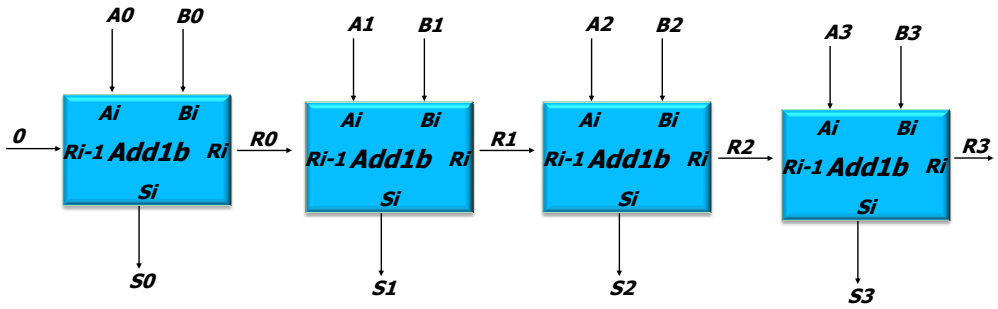


Description structurelle : utilise des composants existant dans une librairie de travail. Le programme instancie les composants et décrit leurs interconnexions.

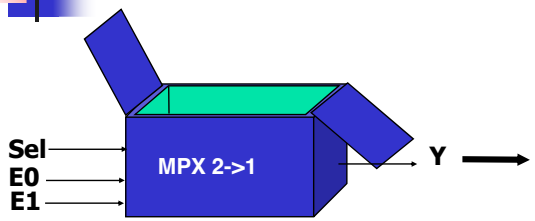
```
ENTITY add2bits IS
PORT (     A, B : IN std_logic_vector (1 downto 0);
           S : OUT std_logic_vector (1 downto 0);
           C : OUT std_logic);
END add2bits;
ARCHITECTURE arch OF add2bits IS
COMPONENT add1b PORT ( Ai,Bi, Rim1 : IN std_logic;  Si, Ri : OUT std_logic);
END COMPONENT;
SIGNAL Masse, Ret : std_logic;
BEGIN
           Masse <='0';
           U1 : add1b PORT MAP (A(0), B(0), masse, S(0), Ret);
           U2 : add1b PORT MAP (A(1), B(1), Ret, S(1), C);
END arch;
```


Description structurelle

Donner la description, sous forme d'équation booléenne, d'un additionneur 1 bit cascadable.
Puis proposer une description structurelle pour un additionneur 4 bits.



Description comportementale



Si Sel= 1
Alors Y= E0;
Sinon Y= E1;

Description comportementale : comprend des blocs d'algorithmes séquentiels exécutés par des processus indépendants. Elle peut traduire un fonctionnement séquentiel ou combinatoire.

```
ENTITY MPX2 IS
PORT (     E0, E1, Sel : IN std_logic;
           Y : OUT std_logic);
END MPX2 ;
ARCHITECTURE arch OF MPX2 IS
BEGIN
           PROCESS (sel,E0,E1)
           BEGIN
                      IF sel='0'   THEN Y <= E0;
                                   ELSE Y <= E1;
                      END IF ;
           END PROCESS ;
END arch;
```

Description ?

Description algorithmique du composant : attention ce n'est pas du C ! Ou tout autre langage.
=> Pensez circuit

S <= A+B;
Architecture ? Additionneur à retenue anticipée, propagée ?

S1<= A+B;
S2 <= C+D ; Architecture ?

Comparez avec le programme C :
S1= A+B;
S2 = C+D;

Plan

1 – Les différents types de description en VHDL
2 – Syntaxe de base
4 – Les instructions concurrentes
5 – Les instructions séquentielles

Eléments de syntaxe : Identificateurs et valeurs littérales

Les commentaires :

Les commentaires débutent par un double tiret.
-- déclaration de signaux interne

Les identificateurs :

Ils se composent d'une suite de lettres ou de chiffres (et éventuellement du caractère '_') débutant par une lettre : A, Horloge1, DATA_OUT.
VHDL ne fait pas de différence, pour les identificateurs et les mots-clés entre majuscules et minuscules.

signal CLK : **bit;** et **SIGNAL** Clk : **BIT**; -- sont identiques.

Les littéraux

Les littéraux sont utilisés pour la représentation de valeurs.
Les plus fréquents :

Entiers : 12 et 0.
Bits : '0' et '1';
Chaines de bits "0110_1110" (le caractère _ est sans action).

Quelques éléments de syntaxe : Identificateurs et valeurs littérales

Les littéraux (suite)

Binaire : Bus <= "1001" ;
Hexadécimale : Bus <= X'9' ; (dans certains outils de synthèse on trouve : 16#9#)

Chaînes de caractères : "cy122ilk".
Concaténation de chaînes : '1' & "10" & '0' équivaut à "1100".

Exo 1 : S<= A (7 downto 0) & B(4 downto 1) & "11 ". Déterminer la taille de S ?
Exo 2 : Décrire un additionneur 4 bits
Entrées : R_{-1}, A, B, Sorties : S, Carry.
Exo 3 : Décalage (gauche ou droite) : concaténation ?

Remarque : la base décimale ne peut pas être utilisée lors de l'affectation des signaux. On peut seulement l'utiliser avec des opérateurs comme le + ou le -.

⇒ ~~A <= 4~~ ; A <= B+4;

Les Types

VHDL : langage fortement typé. Tout objet doit avoir un type défini avant sa création. Le type définit le format des données et l'ensemble des opérations légales sur ces données.

Quatre types :
- les types scalaires : numériques et énumérés (pas de structure interne),
- les types composés : tableau et enregistrement (record) qui possèdent des sous-éléments,
- les types access qui sont des pointeurs,
- le type file qui permet de gérer les fichiers.

A partir d'un type =>possibilité de définir un sous-type par adjonction d'une contrainte et/ou d'une fonction de résolution.
Contrainte : restriction des valeurs possibles à un domaine plus limité que celui qui définit le type de base.

```
SUBTYPE int1 IS INTEGER RANGE expr1 TO expr2;        --(expr2>expr1)
SUBTYPE int2 IS INTEGER RANGE expr3 DOWNTO expr4;    --(expr3>expr4)
```
Exemple : entier de 0 à 255 => *INTEGER RANGE* 0 *TO* 255

Un sous type hérite des opérations définies sur le type de base mais le contrôle du respect de la contrainte sur le résultat est à la charge du programmeur.
Exemple : 200 + 200 = ?

Les types prédéfinis scalaires

Nom du type	Définition de l'ensemble
BIT	Deux valeurs possibles '1 ' ou ' 0 '
BOOLEAN	Deux valeurs possibles True ou False
CHARACTER	Caractères a, b, c …, 1, 2 …
INTEGER	Entiers (signés) sur 32 bits (-2, …*10^9 … à + 2,…*10^9)
REAL	Réels : -1.0E38 à 1.0E38 A:=2.3 ou 1.4E5 ou 1.4E-5
TIME	Nombre réel de temps : fs, ps, ns, us, ms, sec, min, hr. Codage sur 64 bits. Attention : 45ns => erreur, écrire 45 ns

Type std_ulogic : (std_logic n'a pas U)

'U' Unresolved
'X' Forcing Unknown (synthesizable unknown)
'0' Forcing Low (synthesizable logic '0')
'1' Forcing High (synthesizable logic '1')
'Z' High impedance (synthesizable tri-state buffer)
'W' Weak unknown
'L' Weak low
'H' Weak high
'–' Don't care

Exercice : Corriger les expressions suivantes
Temps := 768sec; X<="1112_0000 "; Y<= 17#ABCD#; Z:= A + .3; T:=3,4;

Conversion de type

- On peut effectuer une conversion de type entre deux types de structures proches.
- Cette conversion n'est autorisée que dans trois cas :
  - Conversion de type à représentation entier ou flottant,
  - Conversion de tableaux.
- Syntaxe :
  Type_vers_lequel_on_veut_convertir(valeur_à_convertir)
- **Exemple :**
```
A,B,C,D : REAL;
I : INTEGER;
A:=B*REAL(I); -- conversion de I en REAL puis *
I:=INTEGER(C*D); -- entier le plus proche du réel C*D
```


Déclaration d'un type scalaire

Déclaration d'un type énumération :

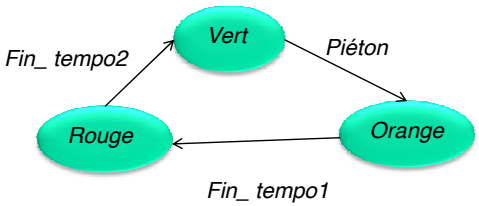
TYPE nom_du _type *IS* (val1, val2, …);

Exemples :

1)TYPE Signal_instruction *IS* (MOVE, ADD, SUB, RTS); ⟶ Définition du type
SIGNAL instr : Signal_instruction; ⟶ Création du signal de ce type

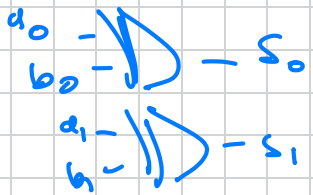
2)Machine d'états

TYPE etat *IS* (Rouge, Orange, Vert);
SIGNAL etat_automate : etat;



Que signifie etat_automate <= etat_automate +1; ou +2 … ?

A OR B => std. logic. keiner.



Les types scalaires

Valeur d'initialisation par défaut :
premier élément de l'énumération ou la plus petite valeur numérique.

Signal_instruction => MOVE,
INTEGER => le plus petit entier négatif.

- **Exercices :**
  - **Exo 1 :** Machine d'états => exemple du feux rouge.
    Déclarer un signal état correspondant à l'automate de deux feux qui sont sur des voies de circulation opposées.
    Quelle sera sa valeur d'initialisation pour le simulateur ?
  - **Exo 2 :** Donner deux déclarations pour un type *index* correspondant à l'indice d'un tableau de 1 à 15.
    Préciser sa valeur d'initialisation.

Trois classes d'objet : constantes, signaux, variables

Déclaration d'une constante :
Valeur non modifiable au cours de l'exécution d'un programme
Déclaration : partie d'une entité, d'une architecture, d'un processus, d'un bloc, d'un paquetage, d'un sous-programme

CONSTANT label **:** type **:=** valeur ;
CONSTANT largeur_du_bus **:** integer **:=** 8;

Déclaration d'un signal :
Correspond aux canaux de communication dans un circuit
Déclaration : partie déclarative de l'architecture (plus rarement, de l'entité, d'un bloc ou d'un paquetage)

SIGNAL nom_du_signal **:** type (**:=** valeur initiale) ;

SIGNAL codeop : std_logic_vector (2 downto 0) ou (0 to 2);
Affectation : codeop <= "10" ou codeop(0) <= '0'.

Autre classe d'objet : constantes, signaux, variables

Déclaration d'une variable
(attention pas de correspondance physique) :
Idem langage de programmation, elles servent de stockage de valeurs nécessaires au bon déroulement d'un algorithme.
Déclaration : processus ou sous-programmes (fonctions ou procédures)

VARIABLE nom_de_la_variable **:** type **:=** valeur initiale ;
VARIABLE nbr_de_boucle : integer := 0 ;
affectation : nbr_de_boucle := 17;

NB : pour les boucles for, il n'est pas nécessaire de déclarer la variable qui sert d'indice de boucle.

Durée de vie des variables :
Fonction : nulle au sens du simulateur,
Processus : du début à la fin de la simulation
=> synthèse ? Problème de portabilité
(une affectation de variable est dynamique, une affectation de signaux est statique ou rémanent)

Autre classe d'objet : constantes, signaux, variables

Exo1 :
Expliquer la différence entre A:=B; et C<=D;
Quelle est le type de A, B, C, D et leur durée de vie ?

Exo2 :
ENTITY bascule **IS**
PORT **(** CLK : ***IN*** std_logic ;
d : ***IN*** std_logic ;
q : ***OUT*** std_logic**)**;
END bascule;
Quelle est le type de CLK, d, Q, leur valeur d'initialisation ?

Affectation

- Affectation à délai nul : A <= B;

Affectation de variables : A:=B;
Affectation de signaux : C<= D;

- Affectation à délai non nul:

A <= B AFTER 10 ns; résultat synthèse ?
C <= '0', '1' AFTER 20 ns, '0' AFTER 40 ns; simulation ?
Dessiner le chronogramme de A.

- ***Autres exemples :***

Q<= "101111"; équivalent à Q<=('1','0','1','1','1','1'); ou
Q<=(5=>'1', 4=>'0',3=>'1',2=>'1',1=>'1',0=>'1'); ou
Q<= (5=>'1', 3 downto 0 =>'1', others =>'0');

Pour mettre tous les bits dans un état : Q <=(others=>'Z'); Rmq : description plus générique

Les opérateurs: de décalage et de rotation

SLL, SRL, SLA, SRA, ROL, ROR.

Ces six opérateurs travaillent sur des tableaux à une dimension de type booléen.

SXL : décalage gauche ou droite logique.
SXA : décalage gauche ou droite arithmétique,
ROX : rotation à gauche ou à droite.

Exemple :

S <= B **SLL** 3 ;

→ "1000 1010" ***SLL*** 3 → "0101 000"
→ "1000 1010" ***SLL*** (-2) → "0010 0010" → "1000 1010" ***SRL*** 2 **??**
→ "0000 1010" ***SLA*** 3 → "0101 0000"
→ "1000 1010" ***SLA*** 3 → "0101 0111"

Exercices :

S0 <= B srl 3; -- Réécrire avec la concaténation sur n bits idem avec sra
S1 <= A sll 2 ; -- Réécrire avec la concaténation sur n bits idem avec sla
S2 <= B rol 3;
"0100 1011" ***SRA*** 3 → ? , "1100 1011" ***SRA*** 3 →
Idem avec les instructions ROL, ROR.

Les opérateurs

Les opérateurs logiques

NOT, AND, OR, NAND, NOR, XOR, XNOR.
Ils sont prédéfinis pour des opérandes de type booléen.

S <= A AND B; Dessiner le schéma pour A, B, S de type std_logic et std_logic_vector.

Les opérateurs relationnels

=, /=, <, <=, >, >=.

Le résultat de ces opérateurs est de type booléen.

Comparer A <= B; et if (A<=B)

Attention : S<=(A=B) Format de S ?

Les opérateurs arithmétiques

+, - , *, **(Exposant), /, MOD(modulo), REM(reste), ABS.

S<= A MOD B; → A modulo B avec le signe de B
S <= A Rem B; → A modulo B avec le signe de A

Seuls les opérateurs +,- sont supportés par les outils de synthèse.

Les opérateurs modulo et reste agissent sur des entiers.

Quartus : pour les autres fonctions → bibliothèque Lpm

Les opérateurs

Les opérateurs arithmétiques (suite)

Lors des opérations : Le type d'entrées des opérandes d'entrée doit correspondre à celui de la sortie.
A+B → A et B de type entier → addition sur des entiers
A+B → A et B de type réel → addition sur des réels (gestion de l'exposant, mantisse …)
Ces opérateurs s'appliquent à des opérandes de booléen (+,-), entier ou flottant.

Pour pouvoir utiliser ces opérateurs pour des opérandes de type std_logic ou std_logic_vector, il faut introduire dans le fichier les bibliothèques suivantes :

```
LIBRARY IEEE;
USE ieee.std_logic_1164.all;
USE ieee.std_logic_unsigned.all;
USE ieee.std_logic_arith.all;
```

Les opérateurs de signe : +,-

- Opérateur unaire
- Rq : les opérateurs de multiplication sont plus prioritaires que les opérateurs de signe: a*-b → erreur; utiliser a*(-b)

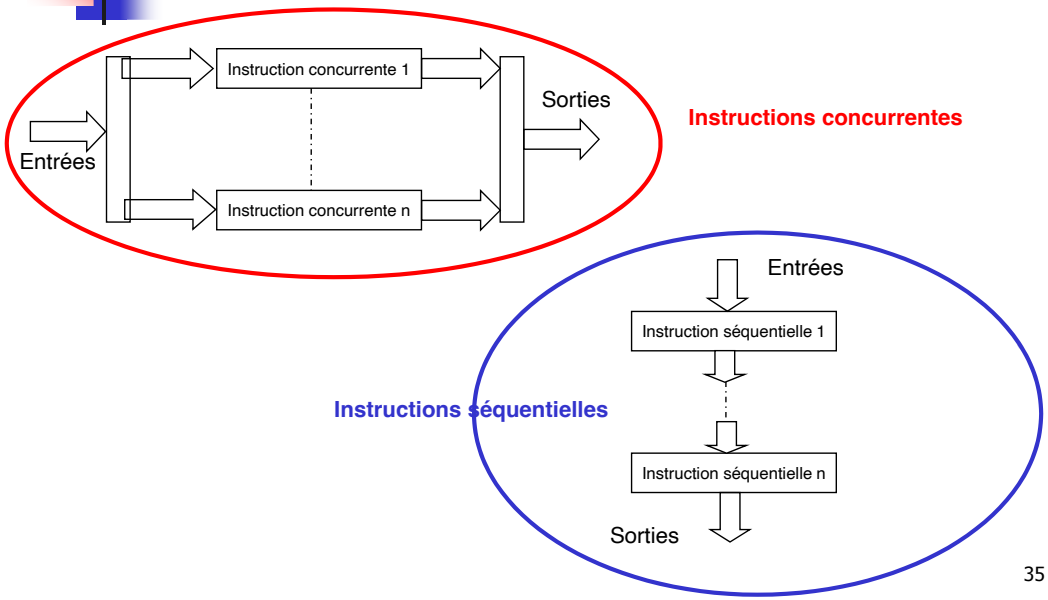
Exercices

- *Exo1 : Préciser la fonction réalisée par les instructions suivantes :*
  - S <= speed >273;
  - S <= T_ext >= T_int; **Type de S ?**
  - S <= (B="00010110");
  - S<=(E=A) ***OR*** (E=D);
- *Exo2 : Comparaison attention*
  - Décrire un comparateur =, >, < pour des données signées et non signées.
- *Exo3 : UAL (ALU)*
  - Décrire une ALU opérant sur des « std_logic_vector ».

Plan

1 – Les différents types de description en VHDL
2 – Syntaxe de base
4 – Les instructions concurrentes
5 – Les instructions séquentielles

Description concurrente ou processus



Description concurrente

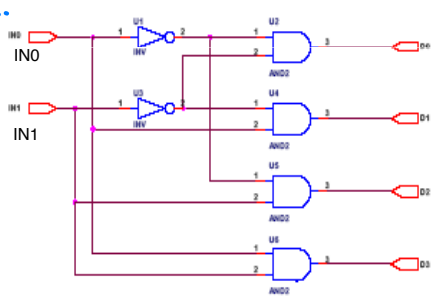
1 - Description concurrente :

Chaque ligne est évaluée indépendamment de l'ordre dans lequel elles sont écrites.
Les instructions sont concurrentes :

les opérations s'exécutent simultanément et en permanence.

U1 : add1b ***PORT MAP*** (a,b,s);
sel <= d ***AND*** c ***AND*** (***NOT*** s) ;
OUT <= x ***WHEN*** (sel='0') ***ELSE*** y ;
MULT : ***PROCESS*** …
END PROCESS MULT;

parallèle.



⇒Exécution simultanée des instructions.

⇒ *Donner la description concurrente du schéma suivant.*

Les instructions concurrentes

1 - Affectation simple ou Equations booléennes

A<=B ; C<= B ***AND*** D; ...

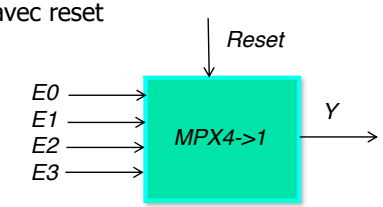
2 - Formes conditionnelles

a – Affectation conditionnelle :

signal1 <= signal2 ***WHEN*** *condition* ***ELSE*** *valeur;*

Exemple : description d'un multiplexeur à 4 entrées avec reset

```
Y <=    '0' WHEN Reset='1' ELSE
        i0 WHEN a='0' AND b='0' ELSE
        i1 WHEN a='1' AND b='0' ELSE
        i2 WHEN a='0' AND b='1' ELSE
        i3 WHEN a='1' AND b='1' ELSE
        'x'; -- indéterminé
```



Les instructions concurrentes

b – Affectation sélective :

WITH expression ***SELECT*** signal1 <= signal2 ***WHEN*** condition1,
....
signaln ***WHEN*** condition n,

Si toutes les conditions ne sont pas spécifiées :
signaln ***WHEN*** others;

⇒ Décrire un MPX 4=>1 avec un bus de sélection Sel avec le *WITH SELECT*.

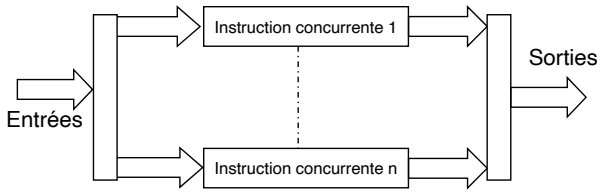
Expliquer la différence entre les instructions conditionnelles et sélectives WITH SELECT et WHEN ELSE

Donner la description :
- MPX, DMPX avec WITH SELECT,
- UAL avec EN avec WHEN ELSE

Plan

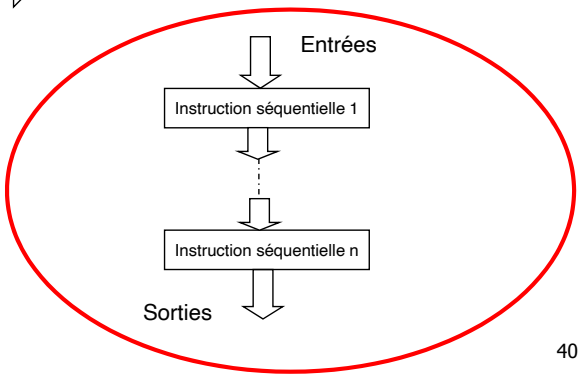
1 – Les différents types de description en VHDL
2 – Syntaxe de base
4 – Les instructions concurrentes
5 – Les instructions séquentielles

Description concurrente ou processus



Instructions concurrentes

Instructions séquentielles (process ou sous-programmes)



Déclaration d'un processus

Les instructions placées dans une zone ***process*** sont exécutées selon l'ordre de présentation (séquentiellement).

Un ***process*** est déclenché à chaque changement d'au moins un des signaux définis dans la liste sensible, située juste aprés le mot clé ***process,*** et encadrée par des parenthèses.

[nom du process] : PROCESS (nom_des_signaux_de_la_liste_de_sensibilité)

BEGIN

suite d'instructions …

END PROCESS [nom_du _process];

Exo : Etablir la liste sensible d'un MPX 4=>1, d'une UAL, d'une bascule J,K simple, d'une bascule JK avec Preset et Clear (synchrone puis asynchrone).

Rmq : 1 seule horloge par process

Les instructions séquentielles : IF THEN ELSE

IF (valeur booléenne) ***THEN*** (suite d'instructions)
ELSIF (valeur booléenne) ***THEN*** (suite d'instructions)
.....
ELSE (suite d'instructions)
END IF ;

La condition implicite pour le ***ELSE*** final est en fait la condition complémentaire de l'ensemble des conditions déjà recensées.

La suite d'instruction est prise en compte que si la valeur booléenne est égale à ***TRUE***.

Exemple :

```
ENTITY dff IS
PORT (  CLK : IN std_logic ;
        d : IN std_logic ;
        q : OUT std_logic);
END dff ;
ARCHITECTURE arch OF dff IS
BEGIN
PROCESS (clk,d ?)
BEGIN
        IF (clk'EVENT AND clk='1')
                THEN q <=d ;
        END IF ;
END PROCESS;
END arch ;
```

Falling_edge(CLK) Ou rising_edge(CLK)

Exo : Donner la description d'un registre (n bits) à chargement parallèle commandé avec Clear et Preset synchrones puis asynchrones avec front montant puis descendant.

Description d'un Compteur

```
ENTITY compteur IS
PORT (    CLK : IN std_logic ;
          CQ : OUT ?? std_logic_vector(7 downto 0);

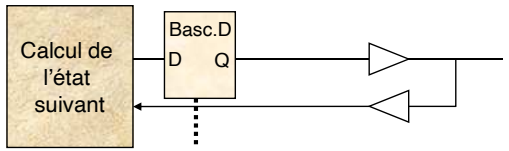
END compteur ;

ARCHITECTURE arch OF compteur IS
BEGIN
PROCESS (clk)
BEGIN
          IF (clk'EVENT AND clk='1')
          THEN    CQ <= CQ + 1;        => Erreur
          END IF ;
END PROCESS ;
END arch ;
```

INOUT ?

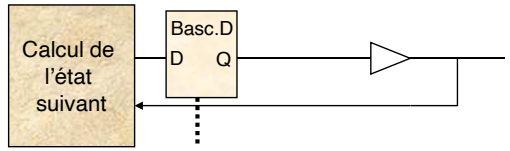
BUFFER ?

Q => INOUT ou BUFFER ?



INOUT

BUFFER




```
ENTITY compteur IS
PORT (    CLK : IN std_logic ;
          CQ : BUFFER std_logic_vector(7 downto 0);
END compteur;
ARCHITECTURE arch OF compteur IS
BEGIN
PROCESS (clk)
BEGIN
     IF (clk'EVENT AND clk='1') THEN   CQ <= CQ + 1; END IF ;
END PROCESS ;  END arch;
```

CQ => Autre possibilité ?

Déclaration d'un signal intermédiaire qui sera copié dans le signal de sortie à la fin du ***PROCESS***.

```
ENTITY compteur IS
PORT (     CLK : IN std_logic ;
           CQ : OUT std_logic_vector(7 downto 0);
END compteur ;
ARCHITECTURE arch OF compteur IS
SIGNAL CMP : std_logic_vector(7 downto 0);
BEGIN
PROCESS (clk)
 BEGIN
     IF (clk'EVENT  AND  clk='1')
          THEN    CMP <= CMP + 1;
     END IF ;
 END PROCESS ;
  CQ<= CMP;
END arch;
```

Quel est le cycle parcouru par ces compteurs ?

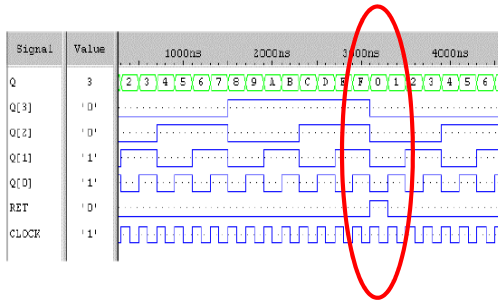
Description d'un Compteur

Exo : Reprendre la description de ce compteur et rajouter les commandes load (synchrone), Clear et Preset (asynchrones).
Rmq : Q<="00000000" ; Q<= (others =>'0');
Rendre générique ce compteur.

Attention : dans un process, les sorties ne sont ré-actualisées qu'à la fin du process.

Exemple :

```
...PROCESS (clk)
BEGIN
          IF (clk'EVENT  AND  clk='1')
          THEN  CQ <= CQ + 1;
                IF  CQ ="1111"
                THEN retenue<='1';
                ELSE retenue<='0';
                END IF;
          END IF ;
END PROCESS ;
```



Quand le signal « retenue » passera-t-il à 1 ?

Remède :

1 -

```
...PROCESS (clk)
BEGIN
          IF (clk'EVENT  AND  clk='1')
          THEN  CQ <= CQ + 1;
                IF  CQ ="1110"
                THEN retenue<='1';
                ELSE retenue<='0';
                END IF;
          END IF ;
END PROCESS ;
```

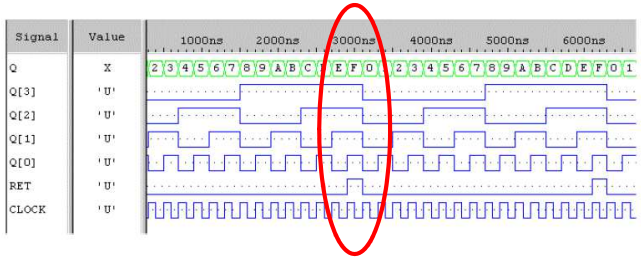
=> Difficilement lisible, synchrone

2 -

```
...PROCESS (clk)
BEGIN
          IF (clk'EVENT  AND  clk='1')
          THEN  CQ <= CQ + 1;
          END IF ;

END PROCESS ;
retenue<='1' WHEN (CQ ="1111") ELSE '0';
```

=> Plus facilement lisible, asynchrone



Les instructions séquentielles : CASE WHEN

Cette syntaxe permet de générer des affectations conditionnelles.

```
PROCESS (...)
BEGIN
CASE expression IS
     WHEN choix 1 => suite d'instructions;
     WHEN choix 2 => suite d'instructions;
          ....;
     WHEN OTHERS => suite d'instructions;

     END CASE ;
```

Attention :

1- ne pas confondre => (implique) et <= (affecte).

2 - Instruction ***NULL*** : l'exécution passe à la ligne suivante.

Exemple

```
ENTITY mux4 IS
PORT (     sel : IN std_logic_vector (1 DOWNTO 0);
           a, b, c, d : IN std_logic ;
           y : OUT std_logic);
END mux4 ;
ARCHITECTURE arch OF mux4 IS
BEGIN
PROCESS (a,b,c,d,sel)
BEGIN
     CASE sel IS
          WHEN "00" => y<= a;
          WHEN "01" => y<= b;
          WHEN "10" => y<= c;
          WHEN "11" => y<= d;
          WHEN OTHERS => NULL;
     END CASE ;
END PROCESS ;
END arch ;
```

Pourquoi ce test ?

Exo : Donner la description d'une UAL, d'un registre à décalage (32 bits).

Déclaration d'un processus

Il est possible de déclarer plusieurs ***PROCESS*** qui sont alors concurrents :

```
nom_process1 : PROCESS (liste sensible1)
        BEGIN .....
        END PROCESS nom_process1;
        nom_process2 : PROCESS (liste sensible2)
        BEGIN .....
        END PROCESS nom_process2; ....
```

Attention :

```
basc_p1 : PROCESS (clk) BEGIN
          IF (clk'EVENT AND clk='1') THEN q<=d;
          END IF;
         END PROCESS basc_p1;
basc_p2 : PROCESS (RAZ) BEGIN
          IF (RAZ='1') THEN q<='0'; END IF;
         END PROCESS basc_p2;
```

Erreur !! Deux (ou plus) processus ne peuvent affecter le même signal. L'état d'un signal est habituellement modifié par un et un seul process.

Néanmoins, les signaux qui disposent d'une fonction de résolution peuvent être modifié par plusieurs process.
La fonction de résolution calcule l'état des signaux en fonction des valeurs imposées par les process agissant sur le signal.

Concurrent/séquentiel (processus)

Attention :

En mode concurrent :

```
a <= b;
c <= b;
a <= 1;  Erreur !!!!!
```

En mode séquentiel :

```
PROCESS (a, b, c)
BEGIN
        a <= b;
        c <= b;  -- Cette description équivaut à a = 1 et c=b
        a <= 1;  -- (pas d'erreur générée).
END PROCESS;
```